GCP Data Engineer

BigQuery Complete Q&A Cheat Sheet

1. Fundamentals and Architecture

Q1. What is BigQuery?

Google Cloud's serverless, fully managed, petabyte-scale data warehouse for analytics (OLAP). You run standard SQL; Google manages infrastructure, scaling and maintenance.

Q2. Explain BigQuery architecture.

Storage and compute are **decoupled**. **Colossus** is the distributed storage layer (columnar format called Capacitor). **Dremel** is the query engine that runs queries as a tree of workers using **slots**. **Jupiter** is the high-speed network, and **Borg** schedules the compute. The in-memory **shuffle** tier moves data between stages.

Q3. Why is BigQuery fast and cost-efficient for analytics?

Columnar storage (reads only needed columns), compression, massively parallel execution, and separation of storage from compute so each scales independently.

Q4. BigQuery vs a traditional data warehouse or RDBMS?

No servers to manage, scales automatically, pay-per-use, handles semi-structured data natively. It is optimized for large scans and aggregations, not for frequent single-row updates or low-latency transactions.

Q5. What are datasets, tables and projects?

Project is the billing and resource container. Dataset is a container for tables, views, routines and models, and is where location and default access/expiration are set. Table holds the data.

Q6. Can you query data across regions?

No. A dataset's location is fixed at creation and cannot be changed. Queries cannot join tables from different locations; copy or replicate the data first (e.g. dataset copy or cross-region replication).

Q7. Standard SQL vs Legacy SQL?

Standard SQL (GoogleSQL) is ANSI-compliant and the default and recommended option. Legacy SQL is the older BigQuery dialect and is not recommended for new work.

2. Storage and Table Types

Q8. What table types exist in BigQuery?

Native (managed) tables, **External** tables (data stays in Cloud Storage, Drive, Bigtable), **BigLake** tables (external data with fine-grained security and caching), **Views**, **Materialized views**, **Snapshots** and **Clones**.

Q9. Active vs long-term storage?

Data in a table or partition that has not been modified for **90 consecutive days** automatically moves to long-term storage at roughly a 50% lower price, with no performance difference.

Q10. Logical vs physical storage billing?

Billing can be based on uncompressed (logical) bytes or compressed (physical) bytes, set per dataset. Physical billing often saves money for highly compressible data, but includes time-travel and fail-safe storage.

Q11. What is time travel and fail-safe?

Time travel lets you query or restore data as of an earlier point (default **7 days**, configurable from 2 to 7 days). **Fail-safe** keeps deleted data for an additional 7 days for Google-assisted recovery only (not user-queryable).

```
SELECT * FROM ds.orders
FOR SYSTEM_TIME AS OF TIMESTAMP_SUB(CURRENT_TIMESTAMP(), INTERVAL 1 HOUR);
```

Q12. Snapshot vs clone?

A **snapshot** is a read-only point-in-time copy (backup). A **clone** is a writable lightweight copy. Both are billed only for data that differs from the base table.

Q13. Sharded tables vs partitioned tables?

Sharded tables are separate tables named by date (events_20260101) and queried with wildcards (_TABLE_SUFFIX). Partitioned tables are preferred: less metadata overhead, better performance, simpler management.

Q14. How does BigQuery handle semi-structured and nested data?
Native **STRUCT** (record) and **ARRAY** (repeated) types plus a JSON type. Denormalizing with nested and repeated fields avoids expensive joins; use UNNEST to flatten.

```
SELECT o.id, i.sku FROM ds.orders o, UNNEST(o.items) AS i;
```

3. Partitioning and Clustering

Q15. What is partitioning and what types exist?
Splitting a table by a column to enable partition pruning. Types: **time-unit column** (DATE/TIMESTAMP/DATETIME; hour, day, month, year), **ingestion time**, and **integer range**. Only one partition column; max **10,000 partitions**.

Q16. What is clustering?
Sorting data within storage blocks on up to **4 columns** (order matters) so BigQuery can skip blocks (block pruning). Supported on STRING, INT64, NUMERIC, BOOL, DATE, DATETIME, TIMESTAMP, GEOGRAPHY, not on FLOAT64, JSON, ARRAY or STRUCT. Reclustering is automatic and free.

Q17. Partitioning vs clustering, and when to combine them?
Partition for coarse, low-cardinality filters (usually date) with an exact cost estimate. Cluster for high-cardinality filter columns (user_id, country); estimate is only an upper bound. Best practice: partition by date and cluster by frequent filter columns.

```
CREATE TABLE ds.events (ts TIMESTAMP, user_id INT64, country STRING)
PARTITION BY DATE(ts) CLUSTER BY country, user_id
OPTIONS (require_partition_filter = TRUE);
```

Q18. What are require_partition_filter and partition expiration?
require_partition_filter makes queries fail unless they filter on the partition column (prevents full scans). Partition expiration auto-deletes partitions older than a set number of days.

Q19. A partitioned-table query still scans everything. Why?
No filter on the partition column, the filter wraps the column in a function or uses a subquery so pruning cannot be resolved up front, or the filter is on another column.

4. Loading, Ingestion and Integration

Q20. Ways to load data into BigQuery?
Batch load jobs from Cloud Storage (free, uses shared slots), **streaming** via the Storage Write API, Pub/Sub BigQuery subscriptions, **Dataflow** pipelines, **Datastream** (CDC from databases), **Data Transfer Service**, DML/INSERT, and the bq CLI or client libraries.

Q21. Which file formats are supported and which is best?
CSV, JSON (newline-delimited), Avro, Parquet and ORC. **Avro and Parquet** are preferred: they are self-describing and efficient; Avro loads in parallel and keeps schema. CSV needs schema handling and is the slowest to parse.

Q22. Batch loading vs streaming?
Batch loading is free and suited to periodic loads. Streaming incurs cost but data is available within seconds. The **Storage Write API** is the recommended streaming method (higher throughput, lower cost, exactly-once semantics) over the legacy tabledata.insertAll.

Q23. How do you design a real-time pipeline to BigQuery?
Source events to **Pub/Sub**, then either a Pub/Sub BigQuery subscription (simple, no transformation) or **Dataflow** (transformations, windowing, dedup) writing to BigQuery with the Storage Write API. Use dead-letter topics for bad records.

Q24. How do you replicate an operational database to BigQuery?
Use **Datastream** for serverless CDC from MySQL, PostgreSQL, Oracle or SQL Server into BigQuery, or Federated queries when data does not need to be copied.

Q25. What is the BigQuery Data Transfer Service?
A managed service that schedules automatic imports from SaaS apps (Google Ads, YouTube), Amazon S3, Teradata, Redshift and other BigQuery datasets.

Q26. External tables, BigLake and federated queries?
External tables query Cloud Storage/Drive data in place (slower, fewer features). **BigLake** adds row/column-level security, metadata caching and multi-cloud/open-format support. **Federated queries** use EXTERNAL_QUERY to read live from Cloud SQL, Spanner or AlloyDB.

Q27. How do you orchestrate BigQuery workflows?
Cloud Composer (Airflow), **Workflows**, **scheduled queries**, **Dataform** (SQL-based transformation with dependencies and testing), or Cloud Scheduler plus Cloud Functions/Run.

5. SQL Features Frequently Asked

Q28. How do you do upserts / CDC in BigQuery?
Use **MERGE** to insert, update or delete in one atomic statement.

```
MERGE ds.target T USING ds.staging S ON T.id = S.id
WHEN MATCHED THEN UPDATE SET T.val = S.val
WHEN NOT MATCHED THEN INSERT (id, val) VALUES (S.id, S.val);
```

Q29. How do you remove duplicates?
Rank with ROW_NUMBER over the key and keep the latest row. **QUALIFY** filters on window functions without a subquery.

```
SELECT * FROM ds.t
QUALIFY ROW_NUMBER() OVER (PARTITION BY id ORDER BY updated_at DESC) = 1;
```

Q30. Common window functions?
ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTILE and running aggregates with OVER (PARTITION BY ... ORDER BY ... ROWS BETWEEN ...).

Q31. What are approximate aggregate functions?
APPROX_COUNT_DISTINCT, APPROX_QUANTILES, APPROX_TOP_COUNT and HyperLogLog++ (HLL_COUNT) functions give fast, cheaper results with small error on huge datasets.

Q32. UDFs, stored procedures and scripting?
UDFs (SQL or JavaScript, persistent or temporary) add custom logic; **stored procedures** (CREATE PROCEDURE) and **scripting** (DECLARE, IF, LOOP, BEGIN...EXCEPTION) enable multi-statement logic and transactions.

Q33. Views vs materialized views?
A view stores only the query and re-runs it each time. A **materialized view** precomputes results, refreshes incrementally and can automatically rewrite queries to use it, giving faster and cheaper reads (with query restrictions). **Authorized views** share a subset of data without granting table access.

Q34. What are search indexes and vector search?
A search index speeds up full-text lookups (SEARCH function) over string and JSON data. Vector indexes support similarity search on embeddings for AI use cases.

6. Performance Tuning and Cost Optimization

Q35. How do you reduce query cost in on-demand pricing?
Select only needed columns (avoid SELECT *), filter on partition and clustering columns, use dry run and **maximum_bytes_billed**, materialize intermediate results, use materialized views, and prefer approximate functions. LIMIT does **not** reduce bytes scanned.

Q36. How do you optimize slow queries?
Filter early, reduce data before JOINs, avoid cross joins and self-joins where window functions work, avoid ORDER BY on huge data without LIMIT, handle **data skew**, denormalize with nested fields, and inspect the **query execution plan** (stages, slot time, shuffle bytes).

Q37. Explain query caching.
Results are cached for about **24 hours** per user and are free when reused. Caching is skipped if the query uses non-deterministic functions (e.g. CURRENT_TIMESTAMP), the table changed, or a destination table is set.

Q38. What is BI Engine?
An in-memory acceleration layer that speeds up dashboards (Looker, Looker Studio, Sheets) with sub-second queries on frequently used data.

Q39. On-demand vs capacity pricing, and what are slots?
Slots are units of compute (CPU and memory). On-demand charges per TiB scanned (first 1 TiB per month free). Capacity pricing (Editions: Standard, Enterprise, Enterprise Plus) reserves slots with autoscaling for predictable cost and performance; reservations and assignments allocate slots to projects.

Q40. How do you monitor cost and usage?
Query **INFORMATION_SCHEMA.JOBS** (bytes billed, slot ms, user), use billing export to BigQuery, Cloud Monitoring and quotas/custom cost controls per user or project.

Q41. Best practices for joins?
Filter both sides first, join on same-typed keys, put smaller table on the right where useful (the optimizer broadcasts small tables), avoid joining on expressions, and pre-aggregate before joining.

7. Security and Governance

Q42. How is access controlled in BigQuery?
IAM roles at organization, project, dataset, table or view level (e.g. BigQuery Admin, Data Editor, Data Viewer, Job User, User). Jobs run in the project of the caller, so Job User is needed to run queries.

Q43. Row-level and column-level security?
Row access policies (CREATE ROW ACCESS POLICY) filter rows per user or group. **Column-level security** uses **policy tags** from a Data Catalog / Dataplex taxonomy, and **dynamic data masking** returns masked values to unauthorized readers.

Q44. Authorized views, datasets and routines?
They let you share query results from a dataset with users who have no access to the underlying tables, by authorizing the view, dataset or routine to read the source.

Q45. Encryption in BigQuery?
Data is encrypted at rest by default (Google-managed keys) and in transit. Use **CMEK** with Cloud KMS for customer-managed keys. VPC Service Controls help prevent data exfiltration.

Q46. How do you handle PII?
Classify with **Cloud DLP / Sensitive Data Protection**, tag columns with policy tags, mask or tokenize, restrict with row/column policies, and audit access through **Cloud Audit Logs**.

Q47. Data governance tools around BigQuery?
Dataplex (data catalog, lineage, quality), Data Catalog metadata, **Analytics Hub** (secure data sharing and exchanges) and audit logs.

8. BigQuery ML and Advanced

Q48. What is BigQuery ML?
Create, train and run ML models using SQL (CREATE MODEL, ML.PREDICT, ML.EVALUATE). Supports linear/logistic regression, k-means, boosted trees, deep neural nets, ARIMA_PLUS time series, matrix factorization, imported TensorFlow models and remote models on Vertex AI.

```
CREATE MODEL ds.churn OPTIONS (model_type='logistic_reg', input_label_cols=['churned'])
AS SELECT * FROM ds.training;
```

Q49. What is BigQuery Omni?
Runs BigQuery analytics on data stored in AWS S3 or Azure Blob without moving the data.

Q50. BigQuery Geospatial?
The GEOGRAPHY type and ST_ functions (ST_DISTANCE, ST_WITHIN, ST_GEOGPOINT) support location analytics.

Q51. What is a medallion / layered design in BigQuery?
Organize datasets into layers: raw/bronze (landing), cleansed/silver, and curated/gold (business marts), using separate datasets for access control and cost tracking.

9. Choosing the Right GCP Service

Service	Type	Best for
BigQuery	Serverless data warehouse (OLAP)	Large-scale analytics, BI, ML on SQL

Service	Type	Best for
Bigtable	Wide-column NoSQL	Low-latency, high-throughput key access (IoT, time series)
Cloud Spanner	Global relational (OLTP)	Strongly consistent, horizontally scalable transactions
Cloud SQL / AlloyDB	Managed relational	Traditional OLTP apps (MySQL, PostgreSQL)
Firestore	Document NoSQL	Mobile/web app backends
Cloud Storage	Object store	Data lake, raw files, backups
Dataflow	Stream and batch processing (Beam)	ETL and real-time pipelines
Dataproc	Managed Spark/Hadoop	Lift-and-shift Spark, custom big-data jobs
Pub/Sub	Messaging	Event ingestion and decoupling
Cloud Composer	Managed Airflow	Workflow orchestration

10. Scenario Questions

Q52. Daily 5 TB event table, queries filter by date and user. Design?
Partition by event date, cluster by user_id, enable require_partition_filter, set partition expiration for retention, and create materialized views or aggregate tables for dashboards.

Q53. Your BigQuery bill suddenly spiked. How do you investigate?
Query INFORMATION_SCHEMA.JOBS by bytes billed, find top users and queries, look for missing partition filters or SELECT *, then apply require_partition_filter, maximum_bytes_billed, per-user quotas and move heavy workloads to capacity pricing if needed.

Q54. Dashboard queries are slow for many users. What do you do?
Enable BI Engine, add materialized views or pre-aggregated tables, cluster on filter columns, and rely on result caching.

Q55. Late-arriving and duplicate streaming records?
Partition by event time so late data lands in the right partition, deduplicate with a unique insert ID or the Storage Write API's exactly-once semantics, and run a MERGE or QUALIFY ROW_NUMBER step to dedupe downstream.

Q56. You accidentally deleted or overwrote a table. How do you recover?
Within the time travel window, query or copy using FOR SYSTEM_TIME AS OF, or restore a deleted table with the bq cp snapshot decorator. Take table snapshots regularly for longer-term protection.

Q57. Share data with a partner without giving table access?
Create an authorized view (or use Analytics Hub exchange) exposing only permitted columns and rows, and apply row access policies and column masking if needed.

Q58. Migrate an on-premises warehouse (Teradata/Hadoop) to BigQuery?
Assess and plan, use the BigQuery Migration Service for SQL translation, move data via Storage Transfer Service or Data Transfer Service to Cloud Storage then BigQuery, validate counts and checksums, run in parallel, then cut over.

11. Numbers and Keywords to Remember

Item	Value / Fact
Max partitions per table	10,000
Max clustering columns	4 (order matters)
Long-term storage threshold	90 days unmodified (about 50% cheaper)
Time travel window	Default 7 days (2 to 7 configurable)
Fail-safe	Additional 7 days (Google recovery only)
Query result cache	About 24 hours, free
On-demand free tier	First 1 TiB queried per month
Batch load and export jobs	Free (shared slots)

Item	Value / Fact
Dataset location	Cannot be changed after creation
Preferred load formats	Avro, Parquet, ORC
Preferred streaming API	Storage Write API
Not billed	Cached results, failed queries, dry runs, metadata operations

Interview tip: Answer in the order: definition, how it works, when to use it, trade-offs, small example. Always link design choices to cost (bytes scanned), performance and security. Verify pricing numbers and limits in the current Google Cloud docs before an interview, since they change over time.